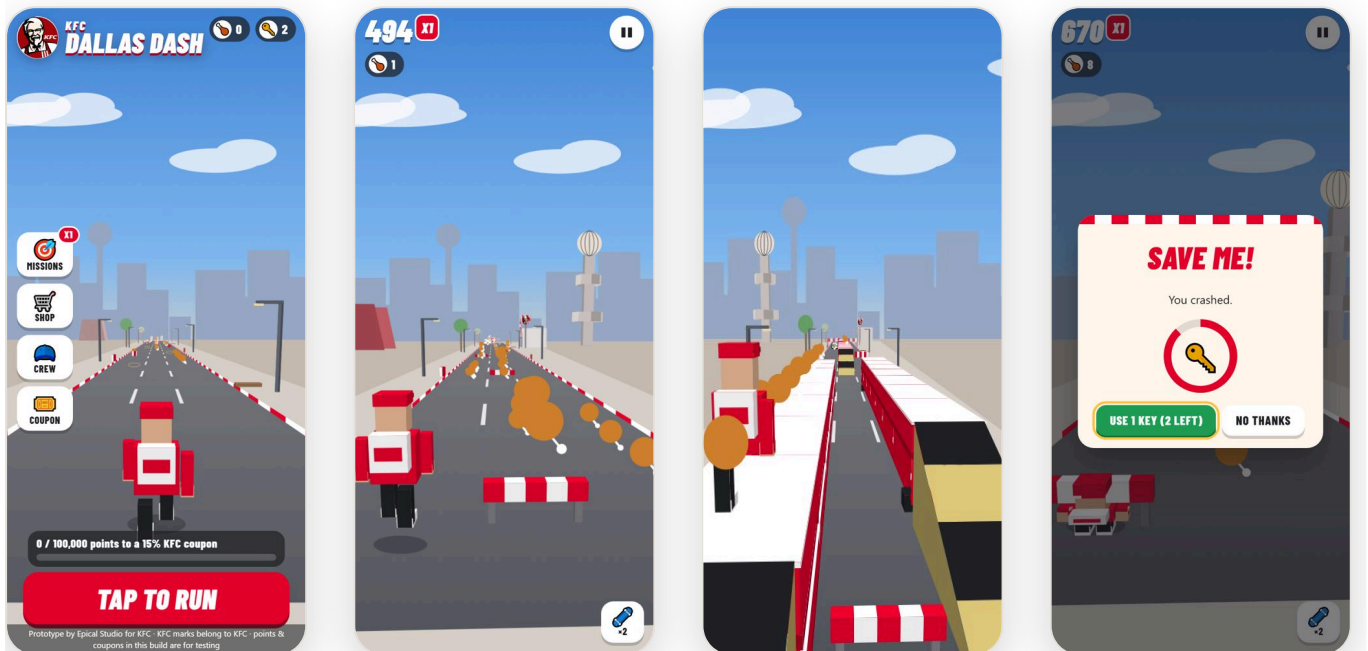


DALLAS DASH

COUPONS, GAME UPGRADE & MOBILE PATHWAY

How a coupon earned in the game becomes a discount in the KFC ordering app with one database, single use, and a unique code every time. What we rebuilt in the game after studying Subway Surfers, and the step-by-step route to iOS and Android with React Native, Expo and EAS from a Windows PC.

1. **Summary:** what was built and what's next
2. **Game → app coupons:** one database, validation, verification, uniqueness
3. **Subway Surfers study:** feature by feature, and what Dallas Dash now does
4. **Mobile pathway:** React Native + Expo/EAS on Windows, phase by phase



Prototype at epical.art/dallas-dash (not indexed). Source in the repo: `dallas-dash/` (game), `coupon-service/` (backend). KFC marks belong to KFC.

1 • SUMMARY

DELIVERED IN THIS ROUND

AREA	WHAT CHANGED	STATUS
Coupon backend	A coupon service with its Postgres schema: issue, validate, reserve, redeem, release, restore and expiry. Codes are random, single-use, owner-bound and stored hashed and encrypted. 7 automated tests run against a real Postgres engine, including two checkouts racing for the same code.	BUILT & TESTED
Game mechanics	The run is now endless and faster: speed climbs from 20 to 46 (it was 14 to 30) and the run no longer stops at 44 seconds. New: rideable trailers with ramps, oncoming vans, stumble and the inspector chase, hoverboard, 4 power-ups with 5 upgrade levels, keys to revive, a drumstick currency, a shop, and missions that raise a score multiplier.	BUILT & TESTED
UI / UX	Subway-Surfers-style flow: a tap-to-run home screen, side menu (Missions, Shop, Crew, Coupon), a minimal in-run HUD with power-up timers, a "Save me!" revive prompt with a countdown, and results with mission progress. Swipes fire mid-gesture, double-tap uses the hoverboard, and the screen shakes and vibrates on hits.	BUILT
KFC branding	KFC red #E4002B with red-and-white bucket stripes, the Colonel logo on the home screen and on the rotating bucket sign, KFC wordmarks on stores, vans and trailers, menu pickups (Original Recipe Bucket, Famous Bowl, Chicken Sandwich and more) and a KFC crew uniform. Page set to noindex.	SWAP IN BRAND KIT
Anti-cheat core	The simulation is now a single shared module. The server replays the same file the phone draws with, so the two can't drift apart. A lookahead bot checks that every course can be beaten.	BUILT & TESTED
Mobile apps	The pathway and stack choice are in section 4. The game logic files carry over to React Native unchanged.	PLAN

Honest limits of the current web build. The prototype at epical.art runs the rules in the visitor's browser, so its points and coupons are for testing only: a determined visitor could edit them. Production needs the server pieces in section 2: the game API replaying runs, and the coupon service on the shared database. The logo is a public reference copy (the Colonel roundel from Wikimedia Commons). Please send KFC's official brand kit (current logo files and typeface) so we can swap it in. Barlow Condensed stands in for KFC's own typeface.

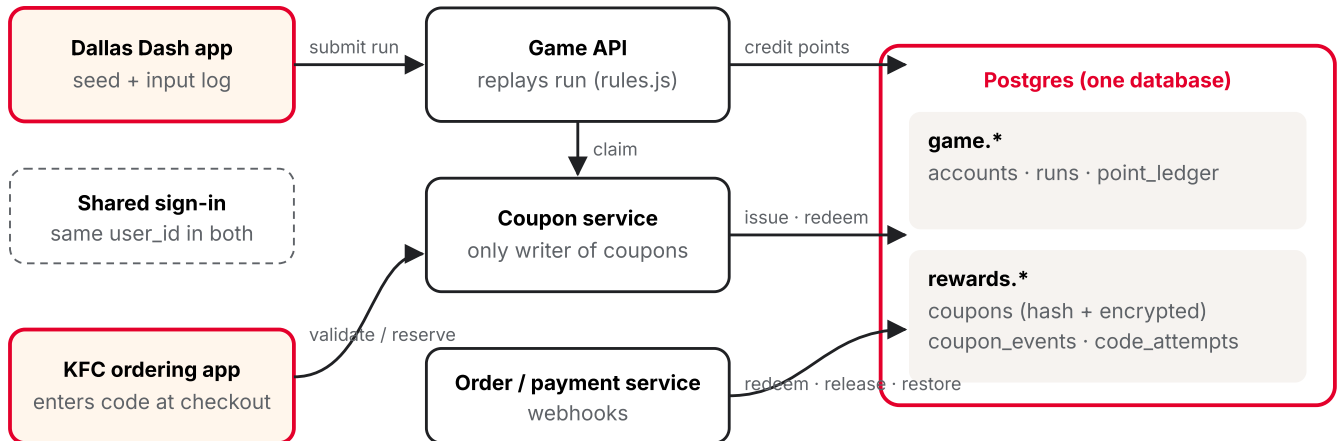
DECISIONS WE NEED FROM KFC

- **Accounts:** which sign-in the KFC app already uses. The game must use the same user IDs, because that is what binds a coupon to a customer.
- **Offer terms:** 15% off, \$25 minimum, \$5 cap, 30 days, 100,000 points are placeholders. Who funds the discount (brand or franchisee), and is it delivery only or pickup too?
- **Order system hooks:** the checkout needs to call `validate / reserve`, and the payment and cancellation events need to call `redeem / release / restore`. We need to know who owns the ordering backend.
- **Legal review** of the reward copy and terms. Dallas Dash is skill-based and needs no purchase, so it is not a sweepstakes, but KFC's counsel should confirm.

2 · GAME → APP COUPONS

YES, ONE DATABASE

One Postgres database (for example Neon, provisioned through the Vercel Marketplace) with two schemas and one owner each. The **game API** writes `game.*`: accounts, verified runs and the points ledger. The **coupon service** writes `rewards.*`: coupons and their event history. The **KFC ordering app never touches the tables**. It calls the coupon service. One writer per table is what makes "a code works once" a guarantee rather than a hope.



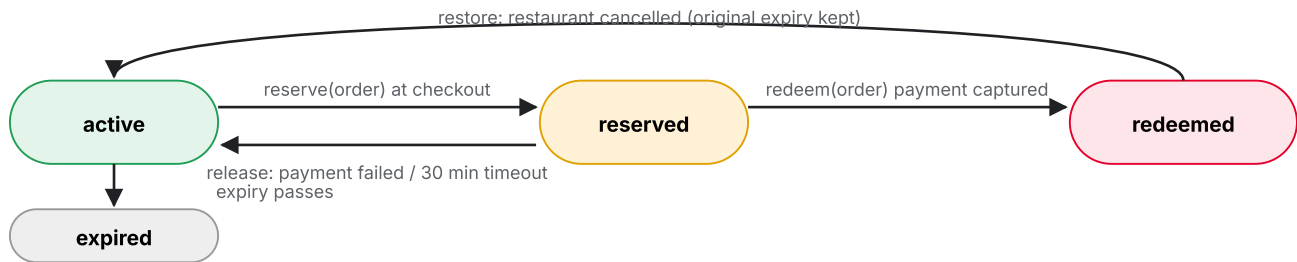
EARNING: POINTS NOBODY CAN FAKE

1. The server hands each account the seed for its next course. A run on any other seed is refused.
2. The phone plays the course and records only the inputs (tick number + swipe), never a score.
3. The game API replays the input log through the same `sim.js` with the account's own loadout from the database (multiplier, boards, keys, upgrade levels), and credits only the score the replay produces. A changed score, a reused `run_id` or more than 6,000 inputs is rejected.
4. Points move only through `game.point_ledger` (+run, -coupon, +refund), so every balance can be audited back to the runs that earned it.

ISSUING: A UNIQUE CODE EVERY TIME

- **Atomic.** The point deduction and the new coupon are written in one transaction, with the account row locked (`SELECT ... FOR UPDATE`). You can never lose points without getting a coupon, or get a coupon without paying the points.
- **Idempotent.** The claim carries an `issueKey`. A double-tap or a network retry returns the same coupon instead of minting two (tested).
- **Random, never derived.** `KFC-7K4Q-9M2P-XW3R-H` is 12 symbols from a cryptographic random generator (60 bits, about 1.15×10^{18} possible codes) plus one check symbol. The alphabet drops I, O, 0 and 1. The code is not computed from the user ID or a counter, so no one can predict the next code. The database's `UNIQUE` index is the final guarantee: on the (practically impossible) clash, the service mints again.
- **Typo-proof.** The check symbol rejects every single mistyped character (verified exhaustively), and 30 of 31 adjacent swaps, before any database lookup.
- **Stored safely.** The service never stores the code in plain text. It keeps a keyed hash (HMAC-SHA256) for lookup, an AES-256-GCM encrypted copy so the owner can see it again in "My coupons", and the last 4 characters for customer support.
- **Limits.** 100,000 points per code, at most 3 unused coupons per account, 30-day expiry.

VALIDATION AND VERIFICATION AT CHECKOUT



STEP

WHAT THE SERVICE CHECKS

Cart preview

`validate`

The code is well-formed (check symbol). It exists and belongs to *this* signed-in user: someone else's code gets the same "not found" answer as a fake one, so the endpoint can't be used to fish for valid codes. It is active and not expired, and the subtotal meets the minimum. Returns the discount, 15% capped at \$5 (\$30 → -\$4.50). Writes nothing.

Place order

`reserve`

Repeats every check, then locks the code to the order with one conditional update (... WHERE `status='active'`). If two checkouts race for the same code, exactly one wins (tested). Retrying the same order returns the same answer. A partial unique index allows only one game coupon per order.

Payment captured

`redeem`

Called by the order/payment service, not by the phone. Marks the coupon spent and records the discount taken. A webhook delivered twice is harmless.

Payment failed

`release`

Unlocks the code so it can be used again. A cron `sweep` also frees reservations older than 30 minutes, and expires old codes.

Restaurant cancels

`restore`

The coupon comes back with its original expiry, or a 7-day grace period if that date has passed.

FRAUD AND ABUSE CONTROLS

RISK

CONTROL

Fake or inflated scores

Server replay of the input log; the server issues seeds; run IDs are unique; the input count is capped. **NEXT** Plausibility checks (inputs per second, superhuman reaction patterns) and a daily points cap.

Bots and modified apps

NEXT Apple App Attest / Google Play Integrity tokens on run submission. Also flag accounts whose scores jump sharply.

Farming many accounts

Coupons are bound to the account the KFC app signs in with. **NEXT** Verify a phone number before the first claim, and limit coupons per payment card per month.

Guessing or sharing codes

60-bit random codes; owner-bound; 10 wrong codes in 15 minutes locks code entry; codes are stored hashed.

Double spending

Conditional state transitions, one game coupon per order, and only the order service can redeem.

Insider / support

`coupon_events` logs every change; support works from the last 4 characters; a `void` status kills a code.

API

POST /coupons/issue	game app	{ issueKey }	→ { code, percent, expiresAt, ... }
GET /coupons	game + app		→ my coupons
POST /coupons/validate 400/404/410/422/429	KFC app	{ code, subtotalCents }	→ { discountCents }
POST /coupons/reserve	KFC app	{ code, orderId, subtotalCents }	→ { discountCents } 409 in_use
POST /coupons/redeem	order svc	{ orderId }	payment captured
POST /coupons/release	order svc	{ orderId }	payment failed / abandoned
POST /coupons/restore	order svc	{ orderId }	restaurant cancelled

Code: `coupon-service/schema.sql`, `src/service.js` and `src/http.js` (web-standard handlers that mount on Vercel Functions, Hono or Express). `npm test` runs 7 tests on PGlite.

TO GO LIVE

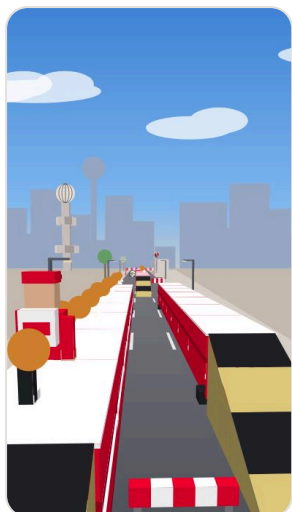
1. Provision Postgres (Neon via the Vercel Marketplace) and run `schema.sql`. Set `COUPON_SECRET`.
2. Deploy the game API: the existing `rules.js` runs server-side unchanged, persisting to `game.*`. Turn `TEST_GRANTS` off.
3. Connect sign-in to the KFC app's identity provider, so both apps share user IDs.
4. KFC app: add a code field at checkout that calls `validate`, then `reserve` when the order is placed. Order service: call `redeem`, `release` and `restore` from the payment and cancellation webhooks. Schedule `sweep` every 5 minutes.
5. Finance reporting: use `coupon_events` to report redemptions and discount dollars per store.

3 • SUBWAY SURFERS STUDY

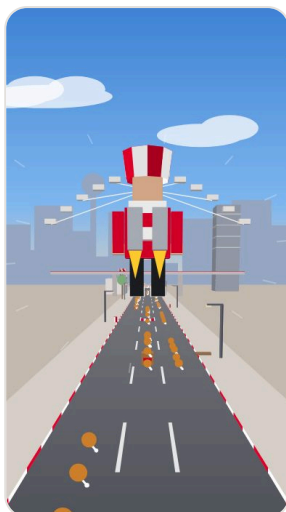
Subway Surfers is the reference endless runner: three lanes, swipe to dodge, jump and roll, trains to run on, and the inspector and his dog chasing you. Around that loop sit power-ups, a hoverboard shield, coins as currency, keys to revive, missions that raise a score multiplier, and rotating "World Tour" city themes. We mapped each feature to Dallas Dash. A few numbers deliberately differ, because here points turn into real money off food.

SUBWAY SURFERS	HOW IT WORKS THERE	DALLAS DASH
Endless run, rising speed	No finish line; speed ramps up until you crash.	BUILT Endless, with a 10-minute cap to keep server replays cheap. Speed goes from 20 to 46, the music tempo and speed lines rise with it, and the difficulty of patterns ramps up over the first ~40 patterns.
Lanes, jump, roll	Swipe left/right/up/down; rolling in the air drops you fast.	BUILT Swipes fire mid-gesture rather than on release, and you can roll mid-air to dive. Keyboard and gamepad also work.
Trains	Parked trains to dodge, trains with ramps to run on top of, moving trains coming at you.	BUILT KFC trailers with ramps (run along the roof and collect drumsticks there), parked KFC vans you can land on, and oncoming vans. Oncoming vans only spawn where the meeting point is clear, so they are always escapable.
Stumble and the inspector	Clipping something makes you stumble and the inspector closes in. Stumble again before he backs off and you're caught. A head-on crash ends the run.	BUILT The health inspector, with clipboard. Small obstacles and side bumps cause a stumble, and a second within 5 s means caught. Barriers, gates and van fronts cause a crash. The screen edges glow red while he's close.
Hoverboard	Double-tap; lasts 30 s and survives one crash.	BUILT Same: 30 s, breaks on the first crash and clears the obstacles just ahead. Bought in the shop for 300 drumsticks; 2 to start.
Power-ups	Coin magnet, jetpack, super sneakers, 2× multiplier, pogo stick. The first four can be upgraded, each level adding 5 s.	BUILT Drumstick magnet, jetpack (flies above the traffic over a trail of sky drumsticks), super sneakers (jump onto vans and trailers), 2× score. 5 upgrade levels at +3 s each. ROADMAP Pogo stick.
Coins	Collected currency for upgrades, boards and characters.	BUILT Drumsticks, laid in lines and in arcs over obstacles: the arc itself shows you where to jump.
Keys and "Save me"	Spend keys to continue after a crash; the cost doubles each time in a run.	BUILT A "Save me!" prompt with a 4-second ring. Costs 1, 2, 4... keys. Keys come from mission sets or cost 1,200 drumsticks. Revives are part of the replayed log, so they are verified too.
Missions → multiplier	Sets of 3 missions; each set raises a permanent score multiplier (up to x30).	BUILT 5 rotating sets that get harder each cycle. Each set gives +1 multiplier, +1 key and +250 drumsticks. Capped at x6 , because the multiplier speeds up coupon earning.
Characters and boards	Unlockable characters and boards, many seasonal.	BUILT 5 KFC crew outfits unlocked by lifetime points. ROADMAP More boards and seasonal crew.
Daily challenges and hunts	Daily word hunt / login rewards; weekly hunts.	ROADMAP "Daily Bucket": spell K-F-C from letters in the run for a daily drumstick bonus. A 7-day streak calendar.

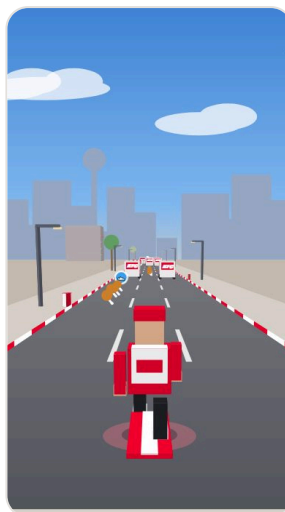
Mystery boxes	Random rewards from missions and hunts.	ROADMAP Only with clearly published odds, and never containing coupon points.
World Tour	Every few weeks a new city theme with its own visuals.	ROADMAP The engine already places named landmarks (10 real Dallas buildings). Next themes could be Houston, Austin or San Antonio for regional KFC campaigns.
Leaderboards	Friends and weekly top scores.	ROADMAP Weekly leaderboard per city. Server-verified scores make it trustworthy.



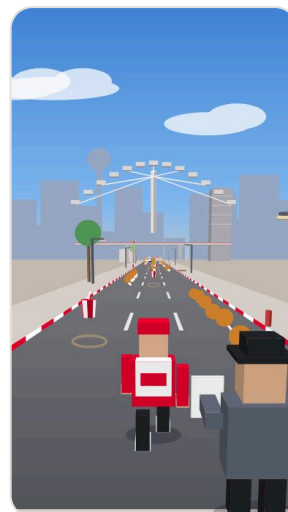
Running on a trailer roof



Jetpack over the traffic



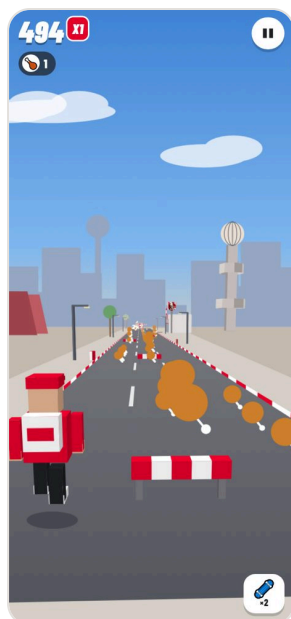
Hoverboard + sneakers power-up



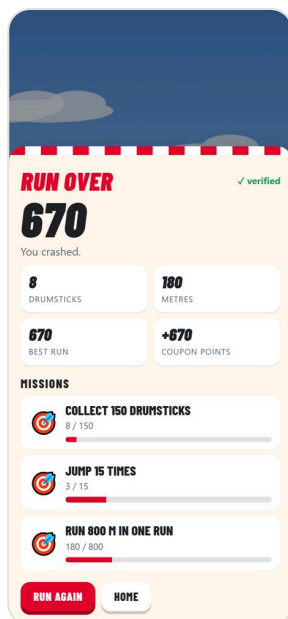
The inspector after a stumble

UI/UX CHANGES

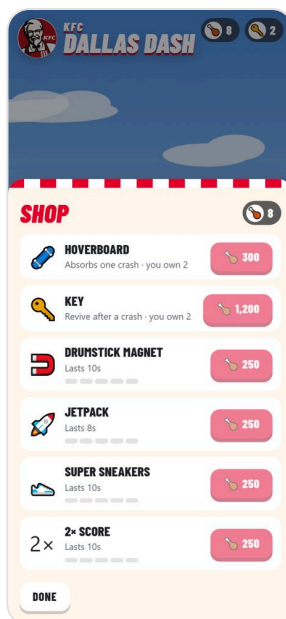
- **Home first, not a form.** Tap-to-run fills the bottom of the screen, the runner jogs in the scene, the menu sits on the left and currencies on the top right, as in Subway Surfers. The coupon progress bar sits right above the play button, so the reward is always visible.
- **Minimal HUD.** Score + multiplier, drumsticks, pause, power-up timers that shrink, and a hoverboard button with its count. The old stat rows and the long explanatory copy were removed from the run screen.
- **Feel.** Faster pace, swipes that fire mid-gesture, screen shake, a white flash and vibration on crashes, a red edge glow during the chase, and pickup toasts named after real menu items.
- **Second chances.** The "Save me!" prompt replaces the old 3-hearts system. That system was forgiving but flat, while one-hit crashes with paid revives create the tension that keeps runners replayable.
- **Progress after every run.** The results screen shows the score, verified coupon points, best run and live mission bars, so there is always a next goal.
- **Accessibility.** Keyboard controls everywhere, focus moves to the primary button in each dialog, reduced-motion is respected, and the touch targets are large.



In-run HUD



Results + missions



Shop



Coupon screen

QUALITY CHECKS RUN

- **Fairness bot:** a lookahead bot played 6 different courses for 3 minutes each (to top speed, about 6.6 km) without crashing or being caught, and an earlier 2-minute pass over 8 courses had zero stumbles. If a perfect player can survive, every pattern is escapable. Each run's full input log replayed to the identical score.
- **Rules tests (5):** verified runs are credited; tampered, replayed and out-of-date runs are refused; revives spend only the keys the account owns; shop prices; coupon threshold, uniqueness and limit; old saves migrate.
- **Browser QA** on a phone and a desktop viewport: full run → crash → revive prompt → results → "✓ verified", with zero console errors. Visual checks of every prop. These found and fixed two renderer bugs: long trailers vanished as you ran along them (no near-plane clipping), and the bridge deck was drawn as a wall across the road (a bug in the original build).

4 • MOBILE PATHWAY (WINDOWS + EXPO/EAS)

RECOMMENDATION: REACT NATIVE WITH EXPO

	REACT NATIVE + EXPO (RECOMMENDED)	FLUTTER
Reuse of the game	<code>sim.js</code> , <code>rules.js</code> and <code>coupon-code.js</code> copy over unchanged. The renderer's draw calls map onto React Native Skia's canvas.	Everything is rewritten in Dart. Worse, the server replays runs with the JavaScript simulation, so you would keep two simulations bit-identical forever, or rewrite the server too.
iOS from Windows	EAS Build compiles and signs iOS in Expo's cloud Macs. Test on a real iPhone via Expo Go or TestFlight.	Possible with third-party cloud Macs or Codemagic, but less integrated.
Updates	<code>eas update</code> ships JavaScript fixes over the air without store review (within store rules).	Store releases (Shorebird exists as an add-on).
Team fit	Same language as the web build, the coupon service and the KFC web stack.	New language.

Current versions at the time of writing: Expo SDK 57 (released 30 June 2026), which ships React Native 0.86. Check expo.dev/changelog before starting.

ARCHITECTURE OF THE APP

```
kfc-dallas-dash/  
  app/                                expo-router screens: (tabs)/home, missions, shop, crew, coupon; run.tsx;  
results.tsx  
  src/game/sim.js                     ← copied unchanged from dallas-dash/  
  src/game/rules.js                   ← only needed offline; the server stays authoritative  
  src/game/coupon-code.js             ← unchanged  
  src/render/GameCanvas.tsx           Skia <Canvas> + useFrameCallback (Reanimated) fixed 60 Hz step  
  src/render/scene.ts                 scene.js ported: ctx.* calls → Skia Path/Paint (same geometry builders)  
  src/input/swipes.ts                 react-native-gesture-handler Pan + double-tap  
  src/api/                             submitRun, claimCoupon, myCoupons (fetch + auth token)  
  src/audio.ts                        expo-audio (short SFX files instead of WebAudio synthesis)
```

Rendering options. (A) Port the renderer to `@shopify/react-native-skia`: recommended, native 60 fps, same look. (B) For a fast MVP, wrap the existing web build in `react-native-webview`: shippable in about a week, but performance and app-store review are weaker. (C) Later, move to real 3D models with `three.js` via `expo-gl` / `react-three-fiber` if KFC wants character models and lighting.

STEP BY STEP ON A WINDOWS PC

1. **Install:** Node.js LTS (22+), Git, VS Code. Optionally Android Studio for the Android emulator. Install **Expo Go** on a real iPhone and Android phone.
2. **Accounts:** an Expo account (free), an Apple Developer Program membership (\$99/year; it must be in KFC's or the franchisee's legal entity name for a branded app), and a Google Play Console account (\$25 one-time).
3. **Create the project:**

```
npx create-expo-app@latest kfc-dallas-dash  
cd kfc-dallas-dash  
npx expo install @shopify/react-native-skia react-native-reanimated react-native-gesture-handler \  
  expo-audio expo-haptics expo-secure-store expo-router  
npx expo start          # scan the QR with Expo Go on the phone
```

4. **Bring the game logic:** copy `sim.js` and `coupon-code.js` into `src/game/`. Port `scene.js` to Skia and the HUD/menus to React Native components.

5. **Cloud builds:**

```
npm i -g eas-cli
eas login
eas build:configure
eas build --profile development --platform android # dev build for native modules
eas build --platform ios # compiled and signed on Expo's Macs; EAS manages certificates
eas submit --platform ios # to App Store Connect → TestFlight
eas submit --platform android # to Play Console internal testing
eas update --branch production # over-the-air JS fixes later
```

6. **Testing without a Mac:** real iPhones via Expo Go during development, then TestFlight builds. Expo's cloud-hosted iOS simulator (EAS Simulator) was in limited preview as of September 2026.

7. **Backend:** deploy the game API and the coupon service (section 2), and point the app at it. Sign in with the KFC app's identity provider. Add App Attest / Play Integrity to run submissions.

8. **Store readiness:** privacy policy and data-safety forms, age rating, reward terms in the app, Sign in with Apple if any social login is offered, and haptics/sound settings. Keep drumsticks and keys earnable only: selling them for money brings in Apple/Google in-app purchase rules.

PHASES

PHASE	TIME	OUTPUT
0 · Decisions	1 week	Brand kit, offer terms, identity provider, ordering-backend owner, legal review started.
1 · Backend live	2 weeks	Neon + game API + coupon service deployed; web build switched to the server ledger; KFC app checkout calls validate/reserve in staging.
2 · App MVP	3–4 weeks	Expo app with the Skia renderer, swipes, sound, haptics, sign-in, run submission, coupon screen; internal TestFlight / Play testing.
3 · Polish & anti-fraud	2 weeks	Device attestation, plausibility checks, performance on mid-range Android (60 fps target), analytics, accessibility pass.
4 · Store launch	1–2 weeks	Store listings, review, phased rollout; daily challenge and leaderboards as the first live update.

Estimates assume one experienced React Native developer plus our team, and a responsive KFC ordering-backend owner. Running costs at launch are small: Apple \$99/year, Google \$25 once, EAS and Neon free or entry tiers (check current pricing), and Vercel hosting for the APIs.

SOURCES

- Subway Surfers power-ups, hoverboard and upgrade levels: Subway Surfers Wiki, subwaysurf.fandom.com/wiki/Power-Ups
- Expo SDK 57 / React Native 0.86: expo.dev/changelog/sdk-57
- Building iOS from Windows with EAS and EAS Simulator: expo.dev/blog/build-ios-apps-on-windows-with-cloud-simulators
- KFC logo reference: Wikimedia Commons, `KFC_logo.svg` (to be replaced with the official brand kit)